

Terraform provisioners

Terraform **provisioners** are a way to execute scripts or commands on a local or remote machine **as part of the resource creation or destruction process**. They are typically used for **bootstrapping, configuration, or custom setup** tasks that cannot be handled by Terraform providers alone.

Provisioners are **not recommended for general use** because they introduce **imperative logic** into Terraform's declarative model. However, they can be useful in specific scenarios like:

- Installing software after VM creation
- Copying configuration files
- Running custom scripts

Types of Provisioners by Execution Phase

Terraform provisioners are categorized based on **when they run** in the resource lifecycle:

1. Creation-Time Provisioners

These run **after the resource is created**, during terraform apply.

Examples:

- file
- remote-exec
- local-exec

Execution Flow:

1. Terraform creates the resource (e.g., EC2 instance).
2. Provisioner connects to the resource (via SSH/WinRM).
3. Executes the defined action (upload file, run script, etc.).

Use Cases:

- Uploading configuration files

- Installing software
- Bootstrapping the VM
- Logging deployment info

2. Destroy-Time Provisioners

These run **before the resource is destroyed**, during terraform destroy.

Syntax:

```

provisioner "local-exec" {
  when    = "destroy"

  command = "echo Destroying ${self.id}"
}

```

Execution Flow:

- Terraform prepares to destroy the resource.
- Executes the provisioner.
- Then destroys the resource.

Use Cases:

- Backing up data before deletion
- Logging or alerting destruction
- Cleaning up external systems
- Triggering de-registration scripts

Type	Triggered During	Common Provisioners	Purpose
Creation-Time	terraform apply	file, remote-exec, local-exec	Setup and configuration
Destroy-Time	terraform destroy	local-exec (with when = "destroy")	Cleanup and teardown

Types of Provisioners in Terraform

Terraform supports **three main types of provisioners**:

1. file Provisioner

Purpose: Copies files or directories from your **local machine** to a **remote resource** (usually a VM).

Use Case: Uploading configuration files, scripts, SSH keys, certificates, etc.

Runs on: Local machine, connects to remote VM via SSH or WinRM.

```
provisioner "file" {  
  source      = "setup.sh"  
  destination = "/home/ubuntu/setup.sh"  
  
  connection {  
    type      = "ssh"  
    user      = "ubuntu"  
    private_key = file("~/.ssh/id_rsa")  
    host      = self.public_ip  
  }  
}
```

2. remote-exec Provisioner

Purpose: Executes commands or scripts **on the remote machine** after it is created.

Use Case: Installing packages, running setup scripts, configuring services.

Runs on: Local machine, connects to remote VM and executes commands there.

```
provisioner "remote-exec" {  
  inline = [  
    "sudo apt-get update",  
    "sudo apt-get install -y nginx"  
  ]  
  
  connection {  
    type      = "ssh"  
    user      = "ubuntu"
```

```

    private_key = file("~/.ssh/id_rsa")
    host        = self.public_ip
  }
}

```

3. local-exec Provisioner

Purpose: Executes commands **on the machine where Terraform is running** (your local system).

Use Case: Logging, triggering scripts, sending notifications, calling external tools.

Runs on: Local machine only.

```

provisioner "local-exec" {
  command = "echo ${self.public_ip} >> ip_list.txt"
}

```

Provisioner	Executes On	Target Machine	Common Use Cases
file	Local machine	Remote VM	Upload files/scripts
remote-exec	Local machine	Remote VM	Run commands/scripts
local-exec	Local machine	Local machine	Log output, trigger tools